Docker Introduction as Environment Versioning

1 Where Docker Fits in Your Existing MLOps Story

Up to now in the course:

- **Git** versions your *code*.
- **DVC** versions your *data* and pipelines.
- **MLflow** versions and manages your *models* and experiments.

There is still a missing piece: the **execution environment**. Even if:

- You check out the same Git commit,
- Pull the same DVC-tracked data, and
- Load the same MLflow model,

you can still get different behavior if:

- Python versions differ (3.8 vs 3.11),
- Library versions differ ("scikit-learn 1.0" vs "1.4"),
- System libraries are missing (e.g., a BLAS implementation), or
- The OS itself behaves differently (Ubuntu vs Windows).

Docker addresses this by letting you *package the entire runtime environment* (OS-level dependencies, system libraries, Python, and your code) into a *Docker image*. Anyone who runs that image as a *container* gets the same environment and behavior.

In other words:

> Git versions code, DVC versions data and pipelines, MLflow versions models, and Docker versions the environment.

2 Core Docker Concepts for MLOps

Lets look at some core concepts about Docker.

2.1 What is Docker?

According to the official documentation, Docker is *"an open platform for developing, shipping, and running applications"* that lets you separate applications from the underlying infrastructure and run them in isolated environments called containers.[1]

Key points to emphasize:

- Docker provides a standard way to **package** and **run** applications.
- It focuses on making applications **portable** and **reproducible** across machines.
- For MLOps, this means your training and inference code can run the same way on laptops, servers, and CI machines.

[1] `https://docs.docker.com/get-started/docker-overview/`

2.2 Docker Engine and Docker Desktop

The Docker platform has two pieces you care about in a local dev / teaching environment:

- **Docker Engine** is the core containerization technology. It follows a client–server architecture:
  - A long-running daemon process (`dockerd`).
  - A REST API used to talk to the daemon.
  - A CLI client (`docker`) that you use in the terminal.[2]
- **Docker Desktop** is a user-friendly application for macOS, Windows, and some Linux distributions that bundles Docker Engine and provides a GUI to manage images and containers locally.[3]

For this course, we will use Docker Desktop.

2.3 Containers

A **container** is an isolated process on the host machine with its own filesystem, which contains everything it needs to run.[4]

Important properties:

- **Self-contained**: the container has all necessary binaries, libraries, and configuration.
- **Isolated**: containers are isolated from each other and from the host.
- **Portable**: you can run the same container image on different machines and get the same behavior.

For MLOps, you can think of a container as "*a repeatable experiment environment*" for training or "*a standardized microservice*" for inference.

2.4 Images

Where do containers get their filesystem and configuration from? From **images**. An image is a standardized, immutable package that includes everything needed to run a container: files, binaries, libraries, and configuration.[5]

Key points:

- Images are **immutable**: once built, they do not change. You create a new image if you want changes.
- Images are typically built from a **Dockerfile**
- For MLOps, you will build images that:
  - Install Python and your ML libraries.
  - Include your training or inference code.
  - Optionally configure connections to MLflow, data locations, etc.

2.5 Containers vs Virtual Machines (VMs)

What's the difference between these two????

- A VM emulates *an entire OS*, with its own kernel and hardware drivers.
- A container is just an isolated process that *shares* the host kernel, but has its own filesystem and dependencies.

[2] `https://docs.docker.com/engine/`
[3] `https://docs.docker.com/desktop/`
[4] `https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/`
[5] `https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-an-image/`

This makes containers:
- Much lighter-weight than VMs.
- Faster to start and stop.
- Easier to scale and replicate.

For MLOps, this means your experiments and services can be spun up very quickly and cheaply.

3 Setting Up Docker

3.1 Installation References

- Official "Get Docker" page (choose your own OS): `https://docs.docker.com/get-started/get-docker/`
- For most of you:
  - Windows and macOS: install Docker Desktop.
  - Linux: install Docker Engine via the distribution's instructions.

3.2 Verifying Docker Installation

Open a terminal and run:

```
docker --version
```

Should see output similar to:

```
Docker version 27.0.3, build xxxxxxx
```

To get more information about the Docker daemon and environment:

```
docker info
```

If Docker is not running, you will get an error. Open Docker Desktop and try again.

4 First Hands-On: Running an Existing Image

This short demo introduces the idea of running pre-built images from registries (e.g., Docker Hub).

4.1 Step 1: Run a Simple Test Container

Run:

```
docker run hello-world
```

- `docker run` tells Docker to run a container based on an image.
- If the image is not present locally, Docker pulls it from a registry (by default, Docker Hub).
- The container runs, prints a message, and exits.

4.2 Step 2: Inspect Containers and Images

Show them how to list running containers:

```
docker ps
```

Likely, nothing is running, because `hello-world` exits immediately. List all containers, including stopped ones:

```
docker ps -a
```

List local images:

```
docker images
```

Remember:

- **Images** are templates (blueprints).
- **Containers** are runtime instances of those templates.

5 Mini MLOps-Aligned Demo: Containerizing a Simple Python Script

In this first lecture, you keep the Python script very simple. Later lectures will plug in MLflow and real training.

5.1 Directory Setup

Create a new directory and move into it:

```
mkdir docker-mlops-demo
cd docker-mlops-demo
```

Create a file `train.py` with the following contents:

```
import platform
import sys

def main():
    print("=== Simple Training Script ===")
    print(f"Python version: {sys.version}")
    print(f"Platform: {platform.platform()}")
    print("Pretend we are training a model here...")
    # Later, this is where MLflow logging would go.

if __name__ == "__main__":
    main()
```

Create a `requirements.txt` file. For now, keep it minimal:

```
# requirements.txt
# Add ML libraries here later (e.g., mlflow, scikit-learn, etc.)
```

5.2 First Minimal Dockerfile

Create a file named `Dockerfile` in the same directory:

```
# Dockerfile: minimal environment versioning example

# 1. Start from an official Python base image
FROM python:3.11-slim

# 2. Set the working directory inside the container
WORKDIR /app

# 3. Copy dependency file and install requirements
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

# 4. Copy the rest of the application code
COPY . .

# 5. Define the default command to run when the container starts
CMD ["python", "train.py"]
```

Explanation:

- `FROM`: chooses a base image (here, a lightweight Python 3.11).
- `WORKDIR`: sets the working directory.
- `COPY requirements.txt` and `RUN pip install`: install Python dependencies inside the image.
- `COPY . .`: copies the rest of the project files into the image.
- `CMD`: tells Docker what to run by default when a container starts.

Each instruction adds a layer to the image.

5.3 Building the Image

From inside `docker-mlops-demo/`, run:

```
docker build -t mlops-train:0.1 .
```

- `docker build` tells Docker to create an image.
- `-t mlops-train:0.1` tags the image with a human-readable name and version (`mlops-train`) and tag (`0.1`).
- The final dot `.` tells Docker to use the current directory as the build context (where it looks for the `Dockerfile`).

After a successful build, list images again:

```
docker images
```

You should see `mlops-train` with tag `0.1`.

5.4 Running the Containerized Script

Now run the container:

```
docker run --rm mlops-train:0.1
```

- `docker run` starts a container from the image.
- `--rm` automatically removes the container after it exits (keeps things tidy for the demos etc).

The output should show something like:

```
=== Simple Training Script ===
Python version: 3.11.x (default, ...)
Platform: Linux-...
Pretend we are training a model here...
```

Highlight:

- This script is now running in a *controlled, versioned environment* defined by the image.
- If you have a different Python version or different host OS, the container's environment is still the same.
- In a future lecture, this script will integrate with MLflow, and you can run it identically on your machine, in CI, or on a server using the same image.

6 Conceptual Wrap-Up

To close the lecture, here are the key conceptual points:

- Docker extends your MLOps stack by making the *execution environment* a first-class, versioned artifact, just like code, data, and models.
- An **image** is the packaged environment; a **container** is a running instance of that environment.
- Docker's client–server model (CLI + daemon) is hidden behind Docker Desktop in local setups, but understanding that model helps when you later move to CI/CD or servers.
- Today's Dockerfile is intentionally minimal; in Lecture 2, you will refine the Dockerfile and dive deeper into image layers and best practices for ML workloads.

In class activity

Do everything below:

- Install Docker
- Recreate the `docker-mlops-demo` project.
- Build and run the `mlops-train:0.1` image.
- Modify `train.py` to print additional environment information (e.g., installed packages) and rebuild the image.

Send me screenshots accordingly on Discord to get points for in-class activity.